What is differnce between Gen-AI and Agentic AI ?

=> Gen is used to created new text,images, video , code from existing pattern

Where Agentic AI can plan,break task into steps , used tools to complete task without human approving.

What is output of LLM ?

=> Probability of possible next word .

What is RAG ?

=> Retrieval Augmented(भर घातलेला) Generation , existing LLM data, add new data (भर घातलेला). So that it give correct answers, instead of guessing.

- Fetching up-dated data, access specific data for response
- No need train to the data.

What problem it solved :

=> 1. Generic model may give incorrect result , but reduce that problem give relevant answers .

2. Reduce the cost and efficiency- existing LLM make fine tune which is expensive , searching data thr whole DB instead it give proper result from specific data.

Why LLM is required for RAG ?

As steps, create chunk, store vector DB, when user asked question, question also converted vector. This vector search in database to find most similar chunk to question .

Then it send prompt(Question + retrieve chunk) to LLM, LLM then refine chunk to get correct answers than hallucination

How it actually works (step by step)

1. **You have a knowledge base** — PDFs, docs, wiki pages, whatever
2. **Chunk it** — split the documents into small pieces (paragraphs, sections)
3. **Embed it** — convert each chunk into a vector (a list of numbers representing its meaning) and store these in a vector database
4. **User asks a question** — e.g. "What's our refund policy for international orders?"
5. **Retrieve** — convert the question into a vector too, then search the vector database for the chunks most similar in meaning to the question
6. **Augment** — take those retrieved chunks and stuff them into the prompt, alongside the user's question
7. **Generate** — send this combined prompt (question + retrieved context) to the LLM, which now answers using the actual retrieved text instead of guessing

.

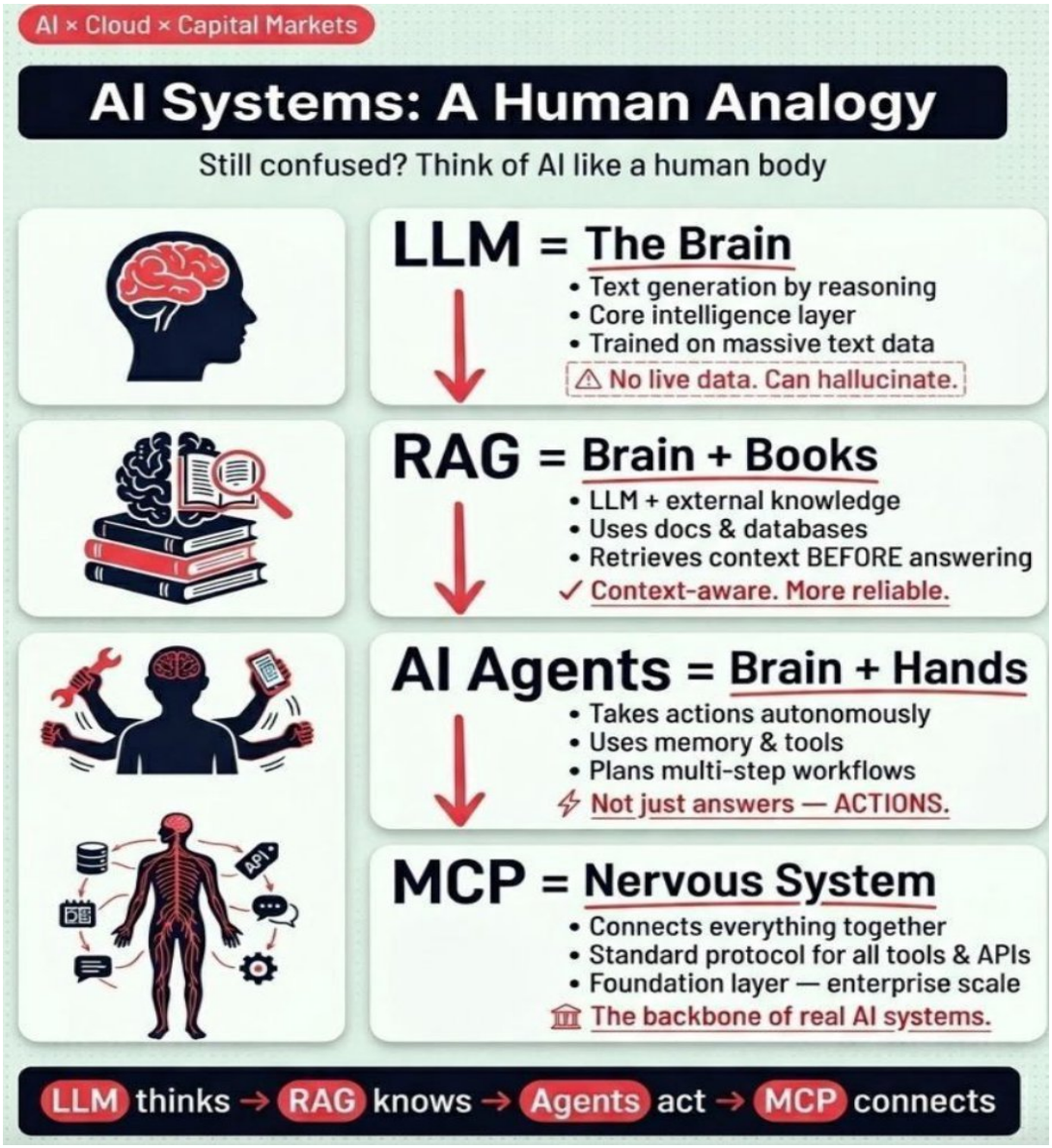
https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/

What is RAG which model you used?

=> For RAG is used Huggingface ,all-MiniLM-L6-v2 model

What is MCP ?

=> Model Context Protocol , It is developed by Anthropic, it is standard framework which help to AI model to connect external resource like github, slack, google drive. It used automate the task. 3 main components , MCP server, MCP client, MCP host.



What is Generative AI ?

=>it is type of AI , used to create new text,images,music, code from learning pattern of existing data. Here training & inference

How Gen AI worked?

=> Gen AI trained on large data set using deep learning network .During the training it learn parameter which help to predict or generate new context.

1. LLM to predict next token
2. RAG - By

What is Few-shot prompt ?

What is Zero-shot prompt ?

=> # Directly asking questions, without giving examples

What is System prompt ?

=>i

What is Few-shot prompting ?

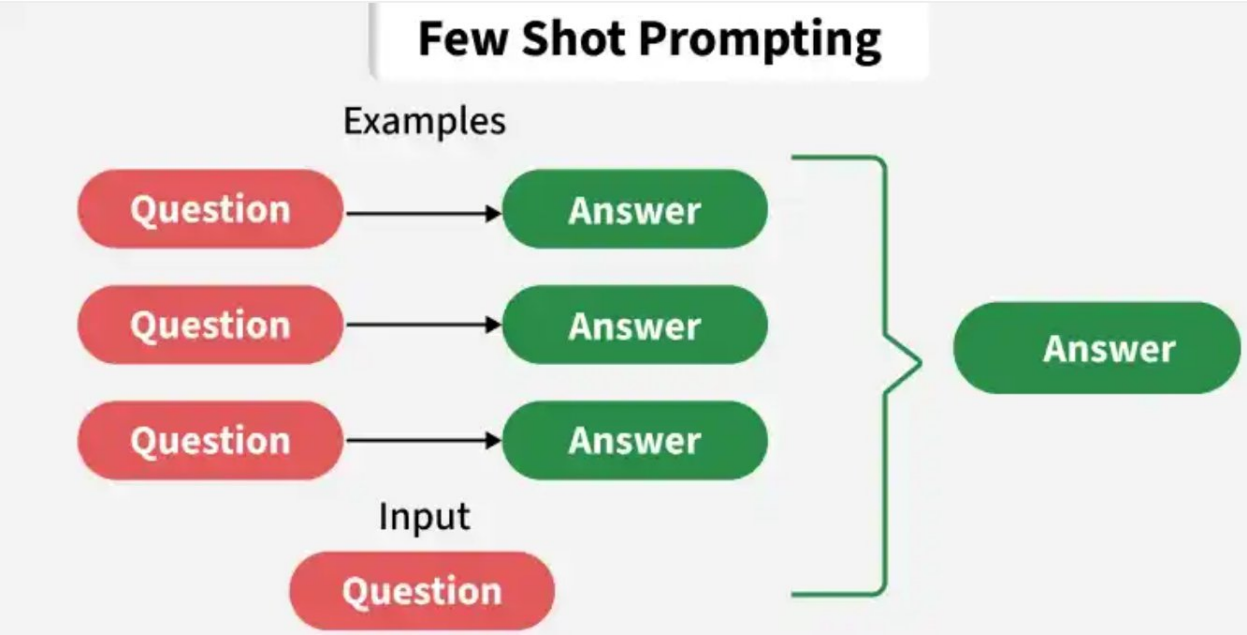
=> It is AI technique where model give few example within prompt to response , It is train from example so it give better response

In-Context Learning (ICL) - Few-shot is just one flavor of ICL. ICL is the bigger idea; few-shot is how you most commonly do it.

You give examples first and then only ask another similar question

Especially if you want answer in a fixed format or any particular type of answer

Always give examples first



What is Vector Embedding ?

=> Computer doesn't understand word, it only ***understand number*** , So converting word(text, images) into number is called as vector embedding, *We need to give similar number to similar word* .Clever part is word with similar meaning kept ***similar*** number list.

Why similar word gives similar numbers ?

=> Similar word gives similar number because it maintain the context.(संदर्भ)

Vector embedding genrated by Mechine learning model , Why?

=> Similar doesn't remain constant ,as its meaning is changed on context.

Ex. If Apple and Phone at first doesn't show any relation but If I make Apple as organization then now they in the same context. That why required vector embedding .

A simple example:

Word	Vector (simplified)
Dog	[0.9, 0.8, 0.1]
Puppy	[0.85, 0.75, 0.12]
Car	[0.1, 0.1, 0.95]

Notice Dog and Puppy have similar numbers → they're "close." Dog and Car are very different → they're "far apart."

What is an Agent?

=> An agent is an AI system that can decide which tools to call and in what order, based on the user's input — rather than following a fixed script.

What is RAG agent ? How it worked?

=> Simi

What are the steps RAG agent using LangGraph ?

=> 1 . RAG agent

1. Text = " user text "
2. Split into chunks
3. Store chunks into vector db .
4. Retrive chunks collection.query(query_texts=[query], n_results=k)

2, LangGraph

1. my_api_key
2. LLM
3. Tools [what ever functions]
4. system_prompt
5. SystemMessage(system prompt
6. create_react_agent
7. Invoke

3. LangChain

1. my_api_key
2. LLM = groq(model, api_key, temperature)
3. llm.invoke
4. prompt (system prompt, user prompt)
5. chain = prompt | llm | parser (

What is difference between RAG and RAG agent ?

=> Real-World Example

Question: "What was the revenue of Apple in 2024 and how does it compare to their 2023 revenue?"

Basic RAG:

1. Searches once → finds random Apple document
2. Might get 2023 data only
3. Returns incomplete answer

RAG Agent:

1. Thinks: "I need 2024 AND 2023 data"
2. Search 1: "Apple revenue 2024" → finds $391 billion
3. Search 2: "Apple revenue 2023" → finds $383 billion
4. Calculates: 2.1% growth
5. Returns complete answer with comparison

In simple terms: RAG is a lookup tool, RAG Agent is a smart researcher that decides what to look up and how many times.

What is chromadb ?

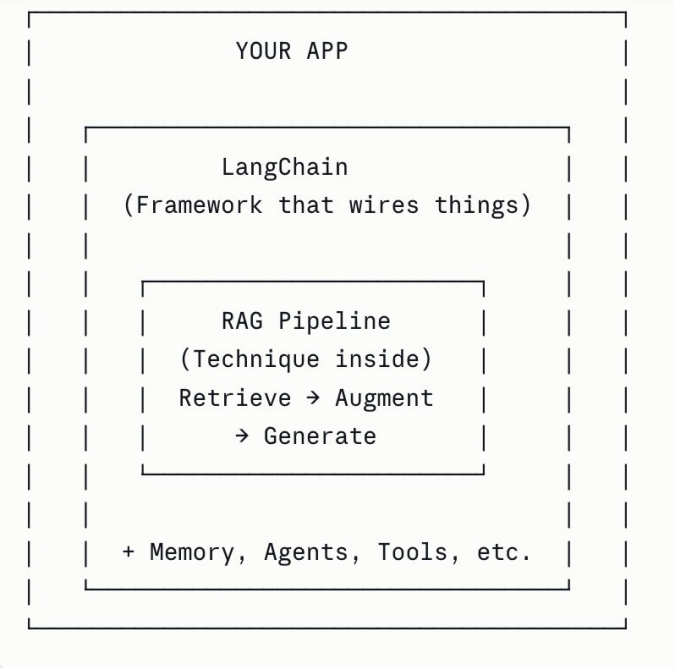
=> it is Vector database used to store generated vector embedding. It simplified the process , you don't need converted into vector embedding and then stored, It did converted into vector embedding and stored.

As from vector DB we rertieve context then why it necessary to again pass to LLM?

=>Vector database provide chunk, but LLM rethink on chunk , remove irrevant chunk , give specific answers of question

What is difference between RAG and langchain and LangGraph ?

=> RAG is technique ,it fetch relevant info before answers. From documents
Langchain is tool/framework , Here you actual predefined steps to task .
Langgraph choose tools , not predefined to automate the task .



What is difference between LLM.invoke and chain.invoke ?
=> LLM invoke directly called what is your query , where chain invoke create template where you can just give topic

What is DocString?
=> It is description given to LLM , it is compulsory. Every tool given to LLM need to give DocString

What are Basic steps for writing agent?
=> 1 . LLM - (model, api, temperature)
2. Tools (function lime multiply,divide)
3. System prompt
4.1. Create_react_agent function
4.2. Query (llm.invoke(‘’message :[(“user”, “what ever your questions”)])

LangChain

What is LangChain ?
=> It is python framework which help to build application using LLM. It help re-useable building blocks like prompt, template, tools, RAG, agent .

Without Langchain – can directly called API.
ex . client = Groq(api_key="YOUR_GROQ_API_KEY")
response = **client.chat.completions.create(**
model="llama-3.3-70b-versatile",
messages=[{"role": "system", "content": "You are a helpful assistant."},

```
{"role": "user", "content": "What is the capital of France?"} ])
print(response.choices[0].message.content)
```

With Langchain

```
llm = ChatGroq( model="llama-3.3-70b-versatile", api_key="YOUR_GROQ_API_KEY")
prompt = ChatPromptTemplate.from_messages([ ("system", "You are a helpful assistant."),
("user", "{question}")])
chain = prompt | llm  ( here it chain, syntax LCEL(LangChain Expression Language) )
response = chain.invoke({"question": "What is the capital of France?"})print(response.content)
```

Imp : Later you want you easily switch from Groq to OpenAI or other LLM

Write steps for summarize text using LangChain?

=> 1. Initialize llm = ChatGroq(model,apiKey, temparature)
2. Initialize prompt using system_prompt, user_prompt
3. Create chain , chain = prompt | llm | StrOutPutParser()
4. Your text = “” “””
5. Response = chain.invoke([“text”: text])

LangGraph

Why LangGraph is required ? Or problem of LangChain?

=> In LangChain step1->step2->step3 are fix , langchain can’t go back if failure, also condition as branching and loop (repeat step), To overcome it required LangGraph.

What is LangGraph? Or what is pillar of LangGraph?

=> LangGraph is open-Source framework. Which consist of State,Node,edges, graph.

1. State : State is data Structure (dictionary/typedict), which pass to every steps (node/edges), Every node can **read / update** state as persistent memory.
2. Node : it is just function , which take current state, do something(mostly call LLM), return update the state.
3. Edges: Give where to go next Node. There are Two kinds of edges:
   1. Normal : always go from A to B, no decision needed.
   2. Conditional edges : decides where to next .
4. Graph : It is object which hold all nodes, edges. It know how run all nodes and edges.

What is StateGraph in LangGraph

=> StateGraph is class inhered from State class , it used to create **blank object** to add notes, edges.it can’t run until **“compile”** the object.

What is add_conditional_edges ?

=> it look current state after it got result ,depending on result it decide which function to run.
Ex.

```
def route_by_intent(state: State) -> str:   # This is same as should_continue
```

```
    if "weather" in state["question"].lower():
        return "weather_tool"
    elif "math" in state["question"].lower():
        return "calculator"
    else:
        return "general_llm"

graph.add_conditional_edges(
    "classify_intent",
    route_by_intent,  # This is same as should_continue, This is branch, having 3 possible path
    {                       # depend on state , it will one of them.
        "weather_tool": "call_weather_api",
        "calculator": "call_calculator",
        "general_llm": "call_llm",
    }
)
```

What is Branching in LangGraph?

=> The decision where to go next depend on current state.

What is @tool?

=> `@tool` becomes callable by the LLM. The **docstring is critical** — the LLM reads it to understand *when* to use the tool.

What is CrewAI?

=> It is open source framework for developing multi Agent system.

==========

Interview questions from claude

Here are some commonly asked **Generative AI (Gen AI) interview questions and sample answers** for freshers and experienced professionals.

1. What is Generative AI?

Answer:

Generative AI is a branch of artificial intelligence that creates new content such as text, images, audio, video, or code by learning patterns from existing data. Unlike traditional AI, which focuses on classification or prediction, Generative AI generates original outputs.

Examples:

- ChatGPT (text generation)

- DALL·E (image generation)
- GitHub Copilot (code generation)

2. What is the difference between AI, Machine Learning, Deep Learning, and Generative AI?

Answer:

Technology	Description
AI	Broad field of creating intelligent systems
Machine Learning	AI systems that learn from data
Deep Learning	ML using neural networks with many layers
Generative AI	Deep learning models that generate new content

3. What is a Large Language Model (LLM)?

Answer:
A Large Language Model (LLM) is a deep learning model trained on massive amounts of text data to understand and generate human-like language.
Examples:

- OpenAI GPT models
- Google Gemini
- Meta Llama

4. How does ChatGPT work?

Answer:
ChatGPT works using a Transformer architecture. It is trained in two stages:

1. **Pre-training**
   - Learns language patterns from large datasets.
   - Predicts the next word in a sequence.
1. **Fine-tuning**
   - Uses human feedback and reinforcement learning.
   - Improves response quality and safety.

5. What is a Transformer Model?

Answer:
A Transformer is a neural network architecture introduced in the paper *Attention Is All You Need*. It uses a mechanism called **self-attention** to understand relationships between words in a sentence.

Advantages:

- Handles long-range dependencies.
- Supports parallel processing.
- Scales efficiently for large datasets.

6. What is Prompt Engineering?

Answer:
Prompt Engineering is the practice of designing effective prompts to get accurate and relevant outputs from AI models.

Example:
Basic Prompt:
Write a summary.
Improved Prompt:
Summarize this article in 100 words using simple language and bullet points.

7. What is Retrieval-Augmented Generation (RAG)?

Answer:
RAG combines:

- Information Retrieval
- Generative AI

The system first retrieves relevant information from a knowledge base and then uses an LLM to generate answers based on that information.

Benefits:

- Reduces hallucinations.
- Provides up-to-date information.
- Improves accuracy.

8. What are Hallucinations in Generative AI?

Answer:
Hallucinations occur when an AI model generates incorrect, fabricated, or misleading information while presenting it confidently.

Example:
An AI inventing a research paper citation that does not exist.

Mitigation Techniques:

- RAG
- Fine-tuning
- Fact-checking
- Human review

9. What is Fine-Tuning?

Answer:
Fine-tuning is the process of further training a pre-trained model on domain-specific data to improve performance for a particular task.

Example:
Fine-tuning a medical chatbot using healthcare datasets.

10. What is Tokenization?

Answer:
Tokenization is the process of breaking text into smaller units called tokens.
Example:
"I love AI"
Tokens:

```
["I", "love", "AI"]
```

LLMs process tokens rather than complete sentences.

11. What is Temperature in LLMs?

Answer:
Temperature controls randomness in generated responses.

Temperature	Behavior
0.0	Deterministic
0.2–0.5	More focused
0.7–1.0	Creative

>1.0	Highly random

12. What is Embedding?

Answer:

Embeddings are numerical vector representations of text, images, or other data that capture semantic meaning.

Applications:

- Semantic search
- Recommendation systems
- RAG systems
- Clustering

13. What is Vector Database?

Answer:

A vector database stores embeddings and enables similarity search.

Popular Vector Databases:

- Pinecone
- Weaviate
- Chroma
- Milvus

14. What is the difference between Fine-Tuning and RAG?

Answer:

Fine-Tuning	RAG
Changes model weights	Does not modify model
Expensive training	Lower cost
Good for behavior changes	Good for knowledge updates
Hard to update	Easy to update

15. What are the limitations of Generative AI?

Answer:

1. Hallucinations
2. Bias in training data
3. Privacy concerns
4. High computational cost
5. Lack of explainability
6. Security risks

16. What is RLHF?

Answer:

RLHF (Reinforcement Learning from Human Feedback) is a training technique where human evaluators rank model responses, helping the model learn preferred behaviors.

Benefits:

- Better alignment with human expectations.
- Safer responses.
- Improved conversational quality.

17. Explain the architecture of a Gen AI application.

Answer:

Typical Flow:

```
User Query
    ↓
Prompt Processing
    ↓
Embedding Model
    ↓
Vector Database
    ↓
Retriever
    ↓
LLM
    ↓
Generated Response
```

18. What evaluation metrics are used for LLMs?

Answer:

- BLEU Score
- ROUGE Score
- Perplexity
- BERTScore
- Human Evaluation
- Accuracy
- Faithfulness

19. What is Context Window?

Answer:

The context window is the amount of information an LLM can process at one time.

Example:

- 8K tokens
- 32K tokens
- 128K tokens

A larger context window allows the model to understand longer documents and conversations.

20. Why is Generative AI important?

Answer:

Generative AI improves productivity by automating content creation, coding, customer support, document summarization, and decision support.

Business Benefits:

- Reduced operational costs
- Faster innovation
- Enhanced customer experience
- Improved efficiency

Interview Question for Experienced Candidates

Q: How would you build a RAG-based chatbot?

Answer:

1. Collect documents.
2. Chunk the documents.
3. Generate embeddings.
4. Store embeddings in a vector database.

5. Retrieve relevant chunks using similarity search.
6. Pass retrieved context and user query to the LLM.
7. Generate the final answer.
8. Monitor accuracy and latency.

This demonstrates practical Gen AI system design knowledge, which is frequently asked in interviews.

Top 5 Questions Most Frequently Asked

1. What is Generative AI?
2. What is an LLM?
3. Explain RAG.
4. Difference between Fine-Tuning and RAG.
5. What are hallucinations and how do you reduce them?

These questions cover the majority of Gen AI interviews for roles such as AI Engineer, Data Scientist, ML Engineer, and Gen AI Developer.

AI × Cloud × Capital Markets

AI Systems: A Human Analogy

Still confused? Think of AI like a human body



LLM = The Brain

- Text generation by reasoning
- Core intelligence layer
- Trained on massive text data

⚠ No live data. Can hallucinate.



RAG = Brain + Books

- LLM + external knowledge
- Uses docs & databases
- Retrieves context BEFORE answering

✓ Context-aware. More reliable.



AI Agents = Brain + Hands

- Takes actions autonomously
- Uses memory & tools
- Plans multi-step workflows

ϟ Not just answers — ACTIONS.



MCP = Nervous System

- Connects everything together
- Standard protocol for all tools & APIs
- Foundation layer — enterprise scale

🏛 The backbone of real AI systems.

LLM thinks → RAG knows → Agents act → MCP connects

Career Clarity

AI Engineer vs Generative AI Engineer

Distinct Roles within AI Development – Uncovering the Differing Job Functions and Tools. A Simple Comparison.

AI ENGINEER

FOCUS

- Train & deploy models to make specific predictions, label data, and identify hidden patterns.

TOOLKIT

- Python
- TensorFlow
- PyTorch
- scikit-learn
- MLOps & ETL Pipelines
- Model Optimization
- ML/DL Frameworks

"Which model should I design and train to most accurately classify this data?"

VS

GENERATIVE AI ENGINEER

FOCUS

- Build applications with Large Language Models (LLMs) to generate new text, code, images, and content.

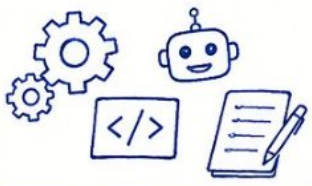
TOOLKIT

- LangChain
- OpenAI API & Libraries
- Vector DBs (e.g., Pinecone, Weaviate)
- RAG Engineering
- Vertex AI & Bedrock

"How can I integrate an LLM to build a specific, helpful user experience?"

QUICK COMPARISON

#	Comparison Point	AI Eng.	Gen AI Eng.
1	Build and train custom models from scratch.	✓	✗
2	Develop solutions using pre-trained LLMs.	✗	✓
3	Implement RAG pipelines and vector databases.	✗	✓
4	Process and utilize large, structured datasets.	✓	✗
5	Architect AI agents and complex workflows.	✗	✓
6	Job Market Demand.	HIGH	VERY HIGH ☆



VISUAL ANALOGY

→ AI Engineer creates the engine and transmission.

→ Generative AI Engineer creates the luxury performance experience that uses it.

- Skills: Multi-agent orchestration, Memory sharing, MCP (Model Context Protocol) server integrations, LangFlow
- Develop advanced RAG (Retrieval Augmented Generation) pipelines with vector databases and semantic search.
-
-
-